```
Created WebConsumer version 1.0.1 with tags main
  Next steps:
    Configure the version branch to be the value of your repository branch.
Pact successfully published for WebConsumer version 1.0.1 and provider MoviesAPI.
  View the published pact at https://mariecruz.pactflow.io/pacts/provider/MoviesAPI/consumer/WebConsumer
/version/1.0.1
  Events detected: contract_published, contract_content_changed (pact content has changed since the last
 consumer version tagged with main)
  Next steps:
    * Configure separate MoviesAPI pact verification build and webhook to trigger it when the pact conte
nt changes. See https://docs.pact.io/go/webhooks
→  pact-js-example git:(main) ✗
```

Figure 6.7 Test output of uploading a new contract to PactFlow

SETTING UP A STATE HANDLER ON THE PROVIDER CODEBASE

The next step is to set up the provider state on the provider codebase. To do this, you need to set up the `stateHandlers` as part of the provider options in the provider/provider-contract.spec.js file. The `stateHandlers` define the different provider states before all the contract verification. The names of the `stateHandlers` must match the names of the provider states defined in the consumer contract test:

```
const options = {
  provider: 'MoviesAPI',
  providerBaseUrl: `http://localhost:${port}`,
  pactBrokerUrl: process.env.PACT_BROKER_BASE_URL,
  pactBrokerToken: process.env.PACT_BROKER_TOKEN,
  providerVersion: '1.0.0',
  publishVerificationResult: true,
  consumerVersionTags: ['main'],
  stateHandlers: {
    'Has a movie with specific ID': (parameters) => {

    }
  }
};
```

Within this state handler, the specific movie ID the consumer uses as part of their contract test must exist before calling the provider verification. There will be multiple ways of doing this, but as part of this code walkthrough, we've kept it simple by getting the first movie and updating its movie ID: